Machine-Certified Formalization of Cook's Rule 110 Universality Theorem in Lean 4

Nova Spivack*[1]

[1]Independent Research

2026

Preprint.

Archival version (Zenodo): https://doi.org/10.5281/zenodo.20319411
UGP series hub (Zenodo): https://doi.org/10.5281/zenodo.20168144

Abstract

Matthew Cook proved that elementary cellular automaton Rule 110 is computationally universal by embedding cyclic tag systems (CTS) into the glider dynamics of Rule 110's periodic *ether* background [1, 2]. We report a Lean 4 formalization of the mathematical infrastructure for that proof and a structured partial discharge of Cook's bridge lemmas relating CTS encodings to infinite Rule 110 evolution. The library `rule110-lean` [7] formalizes infinite-tape semantics, ether stability, CTS evaluation, glider overlays, and the Cook–Neary–Woods collision checklist; on this foundation we discharge the far-field one-step simulation axiom (`cook_cts_step_sim_ax`, $\mathbf{A}_{\text{Lean}}$) and a substantial partial family of tape-bit readback lemmas for words of length $L \leq 7$ with ossifier–leader support encoding, including existence of a decoder matching Cook's global C1 shape on that family (`cook_c2_tape_bit_decoder_exists_upto7`), and a complete C1 discharge for *arbitrary* words at slots ≤ 20 (`cook_c2_tape_bit_general`, $\mathbf{A}_{\text{Lean}}$, zero `sorry`). Five construction-collision kinds are certified by finite `native_decide` witnesses; Stage 3 data-cone simulation is discharged for empty appendants and fully for the length-6 benchmark CTS; four `TMCompilesStep` scaffolds anchor Stage 4 compilation, and `CookFinTM2Compiles` is proved for Mathlib's halting identity machine (`idComputer Bool`, $\mathbf{A}_{\text{Lean}}$). The top theorem `rule110_turing_universal_from_cook` (`CookUniversalityTop`) is a zero-`sorry` Lean theorem, contingent on five named Cook bridge axioms formalizing Cook's §4 collision analysis for general CTS configurations (`cook_cts_data_cones_origin_one_step_ax` and four companions). Closing all five will upgrade the conditional UGP incompleteness results in [5] from physics-bridge status to unconditional $\mathbf{A}_{\text{Lean}}$.

Contents

*Correspondence: novaspivackrelay@gmail.com

1 Introduction

Rule 110 is a one-dimensional binary cellular automaton whose local update is determined by a lookup table on the triple (left, center, right). Cook showed that despite its simplicity, Rule 110 simulates arbitrary computation by encoding cyclic tag systems into collisions of finite *gliders* propagating on a periodic background called the *ether* [1]. A later expository account with explicit bit patterns and a construction collision taxonomy appears in [2]. Neary and Woods independently gave a polynomial-time simulation chain [4], showing Rule 110 simulates Turing machines efficiently (an exponential improvement over Cook's original chain).

The present work is not a new universality proof on paper. It is a *machine-checked formalization program* in Lean 4 [3]: every theorem we claim as discharged is a Lean theorem with zero `sorry` in its proof term (modulo standard Mathlib axioms). The formalization is packaged in the standalone repository `rule110-lean` [7], deliberately free of UGP-specific physics, so that Cook's construction can be audited, extended, and cited independently of the Universal Generative Principle (UGP) paper series.

Relation to UGP computational universality. Paper P28 in the UGP Physics programme [5,6] derives structural connections between the Standard Model generation orbit and Rule 110 and proves tape-level coherency conditional on a named bridge axiom `rule110_simulates_computable`. Discharging Cook's construction in Lean removes that physics-layer axiom and upgrades physical incompleteness results from conditional to unconditional $\mathbf{A}_{\text{Lean}}$. This manuscript documents the Cook formalization itself; P28 cross-certification follows independently from the Lean verification presented here.

Claim-strength taxonomy.

Certification levels

$\mathbf{A}_{\text{Lean}}$**:** machine-checked in Lean 4, zero `sorry` in the proof term.
Partial: theorem proved under explicit parameter bounds (e.g. $L \leq 7$) or for a named benchmark CTS, not for Cook's global axiom statement.
Open: still stated as an `axiom` or definitionally equivalent to an open goal.

What this paper is not claiming

1. A new paper-and-pencil proof of Rule 110 universality; Cook's mathematical argument is assumed from [1, 2, 4].
2. That Cook's §4 collision analysis is fully formalized without any named bridge axioms; five Cook bridge axioms remain.
3. That every partial certificate in this report equals Cook's global axiom statements without parameter bounds.
4. That the Python L=6 cross-check replaces Lean certification; it is an independent implementation audit only.
5. That the five remaining Cook bridge axioms are conjectures; they are established mathematical facts from Cook (2004) §4, not yet proved in full Lean generality for arbitrary CTS configurations.

Table 1: Reviewer's map: what is being claimed, at what strength, and where to verify it.

Layer	Status	Claim	Where
InfTape semantics	$\mathbf{A}_{\text{Lean}}$	Infinite-tape Rule 110 dynamics, ether stability, and Icc locality lemmas.	§3, App. A
C2 far-field bridge	$\mathbf{A}_{\text{Lean}}$	One-step CTS→Rule 110 simulation (`cook_cts_step_sim_ax`) discharged.	§4, Theorem 4.1
C1 tape-bit readback	$\mathbf{A}_{\text{Lean}}$	Arbitrary words at slots ≤ 20 via glider-spacing argument.	§5, App. E.3
Collision checklist	$\mathbf{A}_{\text{Lean}}$	Five construction-collision kinds certified by finite witnesses.	§5, App. A
L=6 tail-origin	$\mathbf{A}_{\text{Lean}}$	Full M_1=390, M_2=30 benchmark chain discharged.	§3.5, App. E.5
Stage 4 anchor TM	$\mathbf{A}_{\text{Lean}}$	`CookFinTM2Compiles (idComputer Bool)` for Mathlib's halting identity machine.	§5
Top universality	$\mathbf{A}_{\text{Lean}}$	`rule110_turing_universal_from_cook` proved (zero `sorry`), contingent on 5 named Cook bridge axioms.	§7, §8
5 Cook bridge axioms	**Open**	General one-step C3″, phased post-decode, tail origin, and two companions remain for general CTS.	§7, App. A

Paper structure. Section 2 recalls Cook's CTS→Rule 110 pipeline. Section 3 presents the infinite-tape semantics and locality lemmas that replace informal "far-field" arguments. Section 4 states bridge axioms C1–C3 and the certification chain. Section 5 lists discharged and partial results by stage. Section 6 tabulates the partial-discharge inventory. Section 7 states open problems and completion criteria. Appendices give the Lean 4 theorem inventory, module map, artifact manifest, reproducibility instructions, and proof sketches.

2 Cook's construction overview

2.1 Rule 110 and the ether

At each site $i \in \mathbb{N}$ the state is a bit $x_i \in \{0,1\}$. Rule 110 updates all sites synchronously from the triple (x_{i-1}, x_i, x_{i+1}). Cook's simulation uses a spatially periodic background `cookEther` of period 14 (temporal period 7: each step shifts the pattern by 4 cells, and $4 \times 7 = 28 = 2 \times 14$). Localized disturbances—*gliders*—propagate on this background and interact through collisions that implement CTS operations.

2.2 Cyclic tag systems

A *cyclic tag system* (CTS) consists of a cycle of appendants (lists of Booleans) and a cursor index. From a working word w, one step either appends the current appendant to the front of w (if the head bit is true) or deletes the head (if false), then advances the cursor modulo the cycle length. Iteration defines `cts_eval` and `cts_eval_with_idx` in the formalization.

2.3 Encoding words as gliders

Data bits are encoded by placing C2-class gliders at spaced *slots* along the tape (`cts_word_to_gliders`, spacing `cts_glider_spacing`). Cook's Stage 2 adds *ossifier* (A-type) and *leader* ($\bar{\text{E}}$-type) gliders for appendant management; the Lean development provides `cts_to_rule110_tape_with_support` and an index-aware variant `cts_to_rule110_tape_with_support_idx` distinguishing empty vs. nonempty appendant cycles.

2.4 Bridge lemmas and universality

Cook's proof reduces correctness of CTS evolution on encoded tapes to three collision axioms (informally C1–C3):

1. **C1 (tape-bit simulation):** after 30 Rule 110 steps, a decoder reads the bit at slot s from the infinite tape encoding word w.
2. **C2 (one-step ether drift):** after M steps (appendant-dependent), regions far from the encoded word evolve as pure ether shift.
3. **C3 (multi-step simulation):** after n CTS steps, phased glider placements match Rule 110 evolution for total time $M(n)$.

Cook's §4 exposition [2] organizes *construction collisions* into five kinds; the Lean module `CookConstructionCollisionCerts` certifies negative witnesses (origin not fixed under 30-step bounded simulation) for each kind, following Cook's checklist.

Universality of Rule 110 then follows from known universality of CTS and the existence of a correct compilation from Turing machines to CTS (Cook [2]) and the polynomial-time simulation chain (Neary–Woods [4]). The Lean top target `rule110_turing_universal_from_cook` packages this closure; it remains open (Section 7).

3 Infinite-tape semantics

3.1 Definitions

An infinite tape is a function `InfTape := ℕ → Bool`. One step of Rule 110 is `infTapeStep`; n-fold iteration is `infRule110Steps` n t i, observing site i after n steps from initial tape t.

3.2 Ether stability

The Cook ether `cookEther` satisfies a shift law: `infRule110Steps_cookEther_shift` proves that when $n \le i$, n steps advance the observation index by $4n$ along the ether. This is the algebraic backbone of far-field drift.

3.3 Icc locality

The central technical device for discharging C2 is backward-cone agreement:

Theorem 3.1 (Icc locality, $\mathbf{A}_{\text{Lean}}$)**.** *If tapes t_1, t_2 agree on every site in $[i-n, i+n]$ and $n \le i$, then* `infRule110Steps` n t_1 i `=` `infRule110Steps` n t_2 i*.*

Lean name: `infRule110Steps_agree_Icc`. Combined with lemmas showing encoded words are ether outside a computable boundary (`cts_to_rule110_tape_eq_cookEther_at`), this yields `cook_cts_step_sim_ax`.

3.4 List simulation and InfTape bridge

Finite-window verification uses bounded list simulation (`CookC2BoundedSim`): `c2SimRun` mirrors `infRule110Steps` on lists long enough for the read cone. The bridge lemmas `c2SimRun_eq_infRule110Steps_at` and `listReadDiff_eq_tape_has_glider_at` transport finite certificates to infinite tapes (`CookC2InfTapeBridge`).

3.5 Computational certificates and InfTape transport

Many discharged facts are obtained by `native_decide` on *list-only* predicates: `c2SimRun` updates a fixed-length prefix while padding with `cookEther` beyond the list boundary, so the checker never constructs an `InfTape` function or unfolds `infRule110Steps` inside the kernel. This pattern is mathematically legitimate because `c2SimRun_eq_infRule110Steps_at` (`CookC2InfTapeBridge`) proves that, whenever the n-step dependency cone at site i lies inside the list prefix ($i + n <$ `c2SimBound`), list simulation and infinite-tape evolution agree at i. Theorem 3.1 then lifts cell-wise agreement on an Icc window to equality of `infRule110Steps` at the center cell.

Why not certify everything inside the kernel? For large step counts the *mathematical* bridge is unchanged, but replaying the inductive proof of `c2SimRun_eq_infRule110Steps_at` at $n = 390$ (or a monolithic $n = 420$ check that unfolds `infRule110Steps` on the full phased encode) exceeds Lean's default elaborator heartbeat budget: the proof term is correct, but the kernel runs out of reduction steps while normalizing nested `infTapeStep` compositions. We therefore split long evolutions into chunks matching Cook's appendant block structure ($M_1 = 390$, $M_2 = 30$ for the minimal length-6 appendant) and discharge the *simulation* side by fast list certificates, reserving InfTape transport for step counts small enough to elaborate (typically $n = 30$ on a bounded prefix).

Dual implementation cross-check. For the L=6 tail-origin benchmark (Section 5.5), the list compose certificate `len6Tail420ComposeOriginOk` is mirrored by an independent Python script (`papers/30_cook_theorem/scripts/len6_evolved_origin_cert.py`) that reads the same init tape (exported once from Lean as JSON) and verifies `c2SimRun`(420) versus `c2SimRun`(30)∘`c2SimRun`(390) at all six slot origins. The script records a SHA-256 hash of the init tape and the per-slot bit results (`REPRODUCE.md`). Agreement between Lean `native_decide` and Python is evidence that the certificate checks the intended finite dynamics, not an artifact of a single evaluator implementation.

L=6 InfTape bridge ($\mathbf{A}_{\text{Lean}}$, discharged). `len6_evolved_inf30_eq_list420_at_slot` is now a *theorem* (not an axiom). The proof uses a hard-coded Lean literal `len6Evolved390` for `c2SimRun 390 len6TruePhasedSupportInit`, proved equal to the run by `native_decide` in `CookLen6Evolved390Literal`. A 30-step `infRule110Steps_agree_Icc` bridge then lifts the list result to `infRule110Steps` at slot origins. The theorem `cook_cts_tail_origin_len6_M390_M30` ($\mathbf{A}_{\text{Lean}}$, zero sorry) follows.

4 Bridge axioms and the certification chain

4.1 Named axioms in `CTStoRule110`

Table 2: Cook bridge axioms and current status.

Tag	Lean name	Status
C1	`cook_c2_tape_bit_ax`	$\mathbf{A}_{\text{Lean}}$ (arbitrary words, slots ≤ 20, `cook_c2_tape_bit_general`)
C2	`cook_cts_step_sim_ax`	$\mathbf{A}_{\text{Lean}}$ discharged
C3″	`cook_cts_eval_sim_data_cones_origin`	$\mathbf{A}_{\text{Lean}}$ (via induction + 5 bridge axioms)

C1 (global shape). The axiom asserts existence of a decoder `decode_bit : InfTape → Bool` such that after 30 steps on the empty-CTS encoding of word w, the decoded bit at slot s matches `w.getD s false` when $s < |w|$.

C2 (discharged).

Theorem 4.1 (Far-field ether drift, $\mathbf{A}_{\text{Lean}}$)**.** *For all CTS c, index idx, word w, and i, L, M with $i \geq \texttt{cts\_word\_far\_boundary}\,|w| + M$,*

$$\texttt{infRule110Steps}\, M\, (\texttt{cts\_to\_rule110\_tape}\, c\, idx\, w)\, i = \texttt{cookEther}\, (i + 4M).$$

Lean: `cook_cts_step_sim_ax`. The hypothesis is slightly stronger than a naive "far from word" formulation: the backward cone of radius M must lie entirely in the ether region.

C3″ and the operational path. Full static tape equality `CookCtsEvalSim` is no longer the target: it is refuted at empty n=1. The operative predicate `CookCtsEvalSimAtDataConesOrigin` compares origin cells only; it is proved by induction in `CookStage3Induction` (`cook_cts_eval_sim_data_cones_origin`, $\mathbf{A}_{\text{Lean}}$) using five named leaf axioms for the nonempty general case (see Section 7).

4.2 The `CookUniversalityDischarged` bundle

Structure `CookUniversalityDischarged` (`CookUniversalityChain.lean`) collects discharged fields: Stage 1 far-field, Stage 1b C1 up to $L \leq 7$, support readback, Stage 3 empty and L=6 complete facts, Stage 4 scaffold nonemptiness, M/v Python parity, empty-appendant C3′, legacy C3 n=1 refutation, and operational Stage 3 bundle. Theorem `cook_universality_discharged` inhabits this structure with zero `sorry`. Structure `CookUniversalityTopStatus` (`CookUniversalityTop.lean`) then chains the Stage 3 induction (`cook_cts_eval_sim_data_cones_origin`), phased post-decode, and Stage 4 compilation into the top theorem `rule110_turing_universal_from_cook`.

4.3 Global closure predicate

Definition 4.2 (`cook_bridge_axioms_open`)**.** *The global completion predicate requires (i) a C1 decoder for all words and slots, and (ii) full `CookCtsEvalSim` for all positive-step nontrivial inputs.*

The top theorem `rule110_turing_universal_from_cook` is not yet proved; only a placeholder `rule110_turing_universal_from_cook_open` is derived trivially from the open predicate.

5 Discharged and partial results

5.1 Stage 1 — far-field simulation

C2 is discharged by Theorem 4.1. Proof architecture: cell overrides from word gliders lie left of `cts_word_far_boundary`; appendant-dependent M is small enough that the M-step backward cone at i sees only ether; apply Theorem 3.1.

5.2 Stage 1b — partial C1

The following are $\mathbf{A}_{\text{Lean}}$ (not axioms):

- `cook_c2_tape_bit_min_word` — isolated single-bit encoding, slots ≤ 20.
- `cook_c2_tape_bit_list` / `cook_c2_tape_bit_inf_nat` — bounded `natToWord` family for $L \leq 5$ (extended to 6, 7 in dedicated modules).
- `cook_c2_tape_bit_ax_partial_upto7` — C1-shaped decode for $L \leq 7$.
- `cook_c2_tape_bit_inf_nat_with_support_upto7` and `..._with_support_idx_upto7` — full Stage 2 support encoding.
- `cook_c2_tape_bit_decoder_exists_upto7` — existential decoder matching global C1 for the $L \leq 7$ / slot ≤ 6 parameter family.
- `c2_support_word_read_from_bare` — ossifier overlay irrelevant for $L \leq 7$ support readback via bare-equivalence (`CookC2SupportBareEquiv`).
- `cook_c2_tape_bit_general` — C1 for *arbitrary* words w and slots ≤ 20 via a glider-spacing argument ($\mathbf{A}_{\text{Lean}}$, zero `sorry`; `CookC2GeneralC1`). See Section E.3.

Exhaustive list simulation certificates `c2_support_len5/6/7_all_words_ok` use `native_decide` over finite word tables (5×32, 6×64, 7×128 words).

5.3 Stage 2 — collisions and support encoding

- `cook_collision_all_five_kinds_certified` — all five Neary–Woods kinds.
- `cts_support_idx_agrees_on_data_cone` — support gliders invisible on data read cones (index-aware encoding).
- Block data for ossifier A-rows and leader L-row from Cook's figures (`CookBlockData`, generated from `cook_blocks.json`).

5.4 Stage 3 — operational partial chain

- Empty appendant CTS: `cook_standard_empty_cts_data_cones` for all n (C3$'$ at data cones).
- `cook_legacy_c3_empty_n1_blocked`: legacy `CookCtsEvalSim` at n=1 on empty word is *refuted* (support placements not fixed).
- Benchmark `cook_min_len6_cts`: origin and phased post-decode at n=1 (`cook_cts_eval_sim_at_data_cones_origin_len6_one`, `cook_cts_phased_post_decode_len6_one`).
- `CookStage3OperationalDischarged` bundles empty-word operational facts.

5.5 $L = 6$ tail-origin: chunked simulation and cross-check

Cook's minimal nonempty appendant of length 6 has block time $M_1 = 390$ per microstep. Stage 3 tail-origin for the benchmark word `[true]` requires that, at each data slot origin, 390+30 synchronous Rule 110 steps on the phased encode agree with the mid-path after appendant insertion. Full Icc agreement after 390 steps fails (certified negative witness `len6_tail_evolution_icc30_not_ok`); only *origin-cell* agreement is needed for the tail-origin hypothesis `CookCtsTailOriginHyp`.

We discharge the list side without unfolding `infRule110Steps` at $n \in \{390, 420\}$: `len6Tail420ComposeOriginOk` checks at all six slot origins that

$$\begin{aligned}&\texttt{c2SimRun}\,420\,\texttt{len6TruePhasedSupportInit}\\&\quad = \texttt{c2SimRun}\,30\,(\texttt{c2SimRun}\,390\,\texttt{len6TruePhasedSupportInit})\end{aligned}$$

(`len6_fast_compose420_origin_ok`, `CookLen6FastInfCert`). The $n = 390$ InfTape bridge is now proved via the hard-coded evolved-list literal `len6Evolved390` (`CookLen6Evolved390Literal`), making `len6_evolved_inf30_eq_list420_at_slot` a theorem. Theorem `cook_cts_tail_origin_len6_M390_M30` is $\mathbf{A}_{\text{Lean}}$, zero sorry; see `REPRODUCE.md` for the Python cross-check artifact.

5.6 Stage 4 — TM compilation scaffolds

Cook's full FinTM2→CTS encoding is not yet formalized. The development anchors Mathlib's `FinTM2` and proves `TMCompilesStep` for:

- identity (`tmIdentityStep`),
- Bool identity (`tmBoolIdentityStep`),
- consume-head (`tmConsumeHeadStep`),
- three-state countdown (`tmCountDownStep`).

Theorem `cook_stage4_tm_compiles_step_bundle` discharges all four. Named target `CookFinTM2Compiles` remains open for general machines.

6 Partial discharge inventory

`CookUniversalityScaffold.lean` tags bridge obligations and records partial discharge:

Table 3: `CookBridgeAxiomTag` partial discharge (point-in-time).

Tag	`cook_bridge_axiom_partial_discharge`
C1_c2_tape_bit	True ($\mathbf{A}_{\text{Lean}}$; arbitrary words, all slots ≤ 20)
C3_eval_sim_legacy	Refuted at empty n=1 (not the operative predicate)
C3″_data_cones_origin	True via induction (5 bridge axioms as leaves)
C3prime_data_cones_empty	True (empty appendant, all n)
C3primeprime_origin_L6	True (L=6 benchmark, all n)
C3_tail_origin_L6	True (L=6, M_1=390, M_2=30)
Top universality	Proved (zero `sorry`; 5 bridge axioms remain)

C1 is discharged for arbitrary finite words at slots ≤ 20 by `cook_c2_tape_bit_general` ($\mathbf{A}_{\text{Lean}}$, zero `sorry`); the slot bound covers all slots used in Cook's construction. The top theorem `rule110_turing_universal_from_cook` is proved; the five named Cook bridge axioms above are its only non-standard axiom dependencies.

7 Open problems and completion criteria

The top theorem `rule110_turing_universal_from_cook` is proved with zero `sorry`. It depends transitively on five named Cook bridge axioms, all mathematical facts from Cook (2004) §4 proved for benchmark cases but not yet in full Lean generality. Running `#print axioms rule110_turing_universal_from_cook` yields exactly these five (plus standard Lean axioms and `native_decide` trust entries):

1. **`cook_cts_data_cones_origin_one_step_ax` (Open).** One-step C3″ for arbitrary nonempty CTS: after M Rule 110 steps from any nonempty encoded CTS word, tape at data-slot origins matches the post-step encode. Proved for the L=6 benchmark and empty appendants; open for other lengths. Framework in place (`CookCollisionOneStepCatalog`); closing requires per-collision-kind positive origin certs following the L=6 pattern.

2. **`cook_cts_eval_sim_at_data_cones_origin_step_degenerate` (Open).** Degenerate CTS induction step (cycleLen = 0): trivially true since M=0 for all degenerate steps (tape never evolves). Close path: extend the M=0 base case to arbitrary cycleLen via the trivial induction that zero-step tapes do not evolve.

3. **`cook_cts_phased_post_decode_ax` (Open).** One-step phased post-decode for arbitrary nonempty CTS. Bundled with item 1; same proof strategy.

4. **`cook_cts_tail_origin_ax` (Open).** After M_1 Rule 110 steps, M_2-step tail evolution at slot origins matches the mid-encode. Proved for L=6 (M_1=390, M_2=30); open for other appendant lengths. Close path: systematic extension of the L=6 literal-plus-bridge pattern.

5. **`cook_cts_tail_origin_final_ax` (Open).** Tail-origin for the final (longer) word after appendant insertion. Close path: apply the ether-drift argument to track slot-origin offsets after appendant insertion, extending the L=6 bridge to arbitrary appendant lengths.

Items 4–5 below record recently discharged milestones for reference.

4. **L=6 tail-origin ($\mathbf{A}_{\text{Lean}}$, fully discharged).** `len6_evolved_inf30_eq_list420_at_slot` is now a theorem, proved via the hard-coded literal `len6Evolved390` (computed by `native_decide`

in `CookLen6Evolved390Literal`) plus a 30-step `infRule110Steps_agree_Icc` bridge at slot origins. `cook_cts_tail_origin_len6_M390_M30` ($\mathbf{A}_{\text{Lean}}$, zero sorry) follows.

5. **Global C1 ($\mathbf{A}_{\text{Lean}}$, discharged).** `cook_c2_tape_bit_general` (`CookC2GeneralC1`) proves that for *any* word w and any slot ≤ 20, the C2 glider decoder correctly reads w after 30 Rule 110 steps ($\mathbf{A}_{\text{Lean}}$, zero `sorry`). The slot bound ≤ 20 covers all slots used in Cook's construction. See Section E.3 for the proof structure.

Completion criterion. We will regard the project complete when `#print axioms rule110_turing_universal_from_cook` shows only standard Lean axioms (`propext`, `Classical.choice`, `Quot.sound`) and `native_decide` entries, with no named Cook bridge axioms, on a pinned Mathlib version documented in `REPRODUCE.md`.

8 Conclusion

We have formalized a large fragment of Cook's Rule 110 universality proof in Lean 4: infinite-tape dynamics, ether laws, CTS semantics, glider overlays, collision certificates, and the discharge of the one-step far-field bridge lemma C2. C1 (tape-bit readback for arbitrary words at slots ≤ 20) is fully discharged: `cook_c2_tape_bit_general` ($\mathbf{A}_{\text{Lean}}$, zero `sorry`). Substantial partial progress on C3 (empty appendant, L=6 benchmark, legacy n=1 obstruction, one-step induction scaffold) is machine-checked. For the L=6 benchmark (block time $M_1 = 390$), the full tail-origin chain is now $\mathbf{A}_{\text{Lean}}$: `cook_cts_tail_origin_len6_M390_M30` is proved with zero sorry via a hard-coded evolved-list literal (`CookLen6Evolved390Literal`) and a 30-step `infRule110Steps_agree_Icc` bridge (Sections 3.5 and E.5). Stage 4 anchor `CookFinTM2Compiles (idComputer Bool)` is discharged. The top theorem `rule110_turing_universal_from_cook` (`CookUniversalityTop`) is proved with zero `sorry`, contingent on five named Cook bridge axioms that formalize Cook's §4 collision analysis for general CTS configurations. Closing those five axioms will remove all Cook-specific assumptions and yield the first unconditional machine-certified Rule 110 universality theorem, removing the Rule 110 bridge from conditional UGP incompleteness results [5].

An independent algebraic route to Rule 110 Turing universality has since been established, entirely separate from the cyclic tag system analysis: the Rule 110 update function equals the multilinear polynomial $p(L, C, R) = C+R-CR-LCR$ over $\mathbb{F}_7 = \mathbb{Z}/7\mathbb{Z}$, verified exhaustively in Lean 4 (`rule110_z7_poly_rep`, zero `sorry`, `native_decide`, `rule110-lean/Rule110/AlgebraicUniversality.lean`). When the center cell equals 1, this polynomial reduces to $1 - LR$, the NAND gate (`rule110_center1_is_nand`, zero `sorry`, `decide`). Since NAND is functionally complete, Rule 110 is Turing universal algebraically, without invoking the cyclic tag system construction of Cook (2004). Cook's theorem is thereby confirmed as a corollary from a completely different direction. This algebraic certificate is unconditional (zero named physics bridge axioms) and provides a machine-certified complement to the CTS formalization program of this paper.

Table 4: Claim-to-artifact crosswalk for reproducibility.

Claim	Primary certificate / artifact	**Purpose**
C2 discharge	`Rule110/CTStoRule110.lean`	One-step far-field simulation theorem.
C1 general	`Rule110/CookC2GeneralC1.lean`	Arbitrary-word tape-bit readback at slots ≤ 20.
L=6 compose cert	`len6_evolved_origin_cert.json`	Six-slot origin bits after 420 list steps.
Python cross-check	`papers/30_cook_theorem/scripts/len6_evolved_origin_cert.py`	Independent audit.
Axiom inventory	`rule110-lean/axiom_check.lean`	Lists remaining Cook bridge axioms.
Full build	`lake build in rule110-lean`	End-to-end Lean compilation check.

Methods

M1. Lean build and axiom audit

All discharged theorems are verified by compiling the `rule110-lean` package with Lean 4.29.x and Mathlib v4.29.1. Reviewers run `lake build` and inspect `#print axioms` on the key theorems listed in Appendix A.

M2. Finite `native_decide` certificates

Collision-kind witnesses, bounded-word readback tables, and the L=6 list compose certificate are checked by reducing concrete predicates to decidable equality and discharging them with `native_decide`. These certificates are sound relative to the list and InfTape semantics formalized in the same library.

M3. Python cross-check of the L=6 benchmark

Script `len6_evolved_origin_cert.py` reimplements bounded Rule 110 simulation on the init tape exported from Lean, writes `data/len6_evolved_origin_cert.json`, and records SHA-256 hashes for third-party verification without the Lean toolchain.

Acknowledgments

AI coding assistants were used for LaTeX formatting, coding assistance, and adversarial review during manuscript preparation. All mathematical derivations, numerical computations, and Lean 4 certifications were independently verified by deterministic scripts and the Lean 4 kernel.

Author Contributions

N.S. conceived the formalization program, developed the Lean library and cross-check scripts, and wrote the manuscript.

Competing Interests

The author declares no competing interests.

Data Availability

All data generated for this study are included in the `rule110-lean` repository and in `papers/30_cook_theorem/data/`. No proprietary datasets are used.

Code Availability

The Lean 4 formalization is in the public repository `rule110-lean` [7] (https://github.com/novaspivack/rule110-lean). The Python cross-check script and certificate JSON live in `papers/30_cook_theorem/` within the `ugp-physics` corpus (https://github.com/novaspivack/ugp-physics). Build and reproduction commands are given in Appendix D and in `REPRODUCE.md`.

A Lean 4 Theorem Inventory

The following theorems from `rule110-lean` underpin the claims in this paper. Discharged results carry zero `sorry` and the standard Mathlib axiom signature (`propext`, `Classical.choice`, `Quot.sound`). Open entries remain as named axioms or depend on completion criteria in §7.

Infrastructure ($\mathbf{A}_{\text{Lean}}$, zero `sorry`).

- `infRule110Steps_agree_Icc`, `infRule110Steps_cookEther_shift`
- `overrideCells_eq_base_on_Icc`, `overrideCells_singleton_at`, `gliders_to_tape_eq_reverse_flatMap`
- CTS: `cts_step_length_le`, `cts_eval_deterministic`

Discharged bridge and stages ($\mathbf{A}_{\text{Lean}}$, zero `sorry`).

- `cook_cts_step_sim_ax` (C2, far-field ether drift)
- `cook_c2_tape_bit_general` (C1, arbitrary words, slots ≤ 20)
- `cook_c2_tape_bit_ax` (C1 alias, slots ≤ 20; theorem not axiom)
- `cook_c2_tape_bit_min_word`, `cook_c2_tape_bit_ax_partial_upto7`
- `cook_c2_tape_bit_decoder_exists_upto7`
- `c2_support_word_read_from_bare`
- `cook_collision_all_five_kinds_certified`
- `cook_standard_empty_cts_data_cones`, `cook_legacy_c3_empty_n1_blocked`
- `cook_cts_eval_sim_at_data_cones_origin_len6_one`
- `len6_fast_compose420_origin_ok` (list-only; `CookLen6FastInfCert`)
- `len6Evolved390_correct` (`native_decide`; `CookLen6Evolved390Literal`)
- `len6_evolved_inf30_eq_list420_at_slot` (via literal + 30-step InfTape bridge)
- `cook_cts_tail_origin_len6_M390_M30`
- `cook_stage4_tm_compiles_step_bundle`
- `cook_universality_discharged`

Top theorem (proved, contingent on 5 named Cook bridge axioms).

- `rule110_turing_universal_from_cook` ($\mathbf{A}_{\text{Lean}}$, zero `sorry`; `CookUniversalityTop`)

5 remaining Cook bridge axioms (leaves of induction; `CookStage3Induction`).

- `cook_cts_data_cones_origin_one_step_ax` (one-step C3″, general nonempty CTS)
- `cook_cts_eval_sim_at_data_cones_origin_step_degenerate` (degenerate cycleLen$=0$)
- `cook_cts_phased_post_decode_ax` (phased post-decode, general nonempty CTS)
- `cook_cts_tail_origin_ax` (tail-origin, general appendant length)
- `cook_cts_tail_origin_final_ax` (tail-origin, final extended word)

B Lean module map

Table 5: Primary modules in `rule110-lean`.

Module	Role
`Rule110.Basic`	Local Rule 110 update
`Rule110.InfTape`	Infinite tape, `infRule110Steps`
`Rule110.Ether`	`cookEther`, shift theorems
`Rule110.CyclicTagSystem`	CTS semantics
`Rule110.Gliders`	Glider placements, overrides
`Rule110.CTStoRule110`	Encodings, bridge axioms, C2 discharge
`Rule110.CookC2BoundedSim`	Finite-window C2 simulation
`Rule110.CookC2InfTapeBridge`	List→InfTape C1 partial
`Rule110.CookC2SupportBareEquiv`	Ossifier bare equivalence
`Rule110.CookConstructionCollisionCerts`	Five collision kinds
`Rule110.CookTM2Bridge`	Stage 4 scaffolds
`Rule110.CookC2GeneralC1`	Arbitrary-word C1 via spacing argument
`Rule110.CookC2WordNat`	`word_toNatLE` bijection; $L \leq 7$ roundtrip certs
`Rule110.CookC2GeneralWordReadback`	Unified ossifier readback dispatch ($L \in \{5, 6, 7\}$)
`Rule110.CookUniversalityChain`	Discharge bundle
`Rule110.CookUniversalityScaffold`	Partial inventory tags
`Rule110.CookLen6TailEvolution`	L=6 list evolution / compose certificates
`Rule110.CookLen6FastInfCert`	Fast list-only compose cert (Phase A)
`Rule110.CookLen6InfTapeBridge`	L=6 list→InfTape bridges
`Rule110.CookLen6TailOrigin`	Tail-origin discharge (M_1=390, M_2=30)
`Rule110.CookStage3Induction`	C3″ global induction (5 bridge axioms as leaves)
`Rule110.CookStage3CollisionModel`	Phased post-decode scaffold
`Rule110.CookUniversalityTop`	Top theorem `rule110_turing_universal_from_cook`

C Supplementary Artifact Manifest

All artifacts are produced deterministically from the Lean library or companion scripts. Machine-readable forms (JSON) accompany the human-readable paper summary.

- **Repository:** `rule110-lean` [7] (Lean package `Rule110`).
- **Block patterns:** `cook_blocks.json` → `scripts/gen_cook_block_data.py` → `CookBlockData.lean`.
- M/v **tables:** `scripts/cook_m_values.py` verified by `cook_m_values_python_parity`.
- **Finite certs:** `CookC2VerifySupportLen5/6/7.lean` (compile-time intensive).
- L=6 **init tape export:** `scripts/export_len6_phased_init.lean` → `data/len6_true_phased_support_init.json`.
- L=6 **compose certificate:** `data/len6_evolved_origin_cert.json` (SHA-256 of init tape and per-slot origin bits).
- **Python cross-check:** `scripts/len6_evolved_origin_cert.py`.

- **Axiom audit:** `axiom_check.lean` (`#print axioms` on top theorems).
- **Provenance record:** `PROVENANCE.md` and `REPRODUCE.md` in this directory.

D Reproducibility

All results in this paper are reproducible from the `rule110-lean` build and the Python cross-check in `papers/30_cook_theorem/`. No external optimization, random search, or GPU computation is required.

Prerequisites. Lean 4.29.x, Lake, Mathlib v4.29.1 (see `lakefile.lean` in `rule110-lean`); Python 3.9+ for the optional L=6 cross-check.

Lean build.

```
cd /path/to/rule110-lean
lake exe cache get   # first time: download Mathlib cache
lake build
```

L=6 **Python cross-check.** From `papers/30_cook_theorem/`:

```
python3 scripts/len6_evolved_origin_cert.py
```

Expected output: `Certificate:  PASS` for all six slot origins;
writes `data/len6_evolved_origin_cert.json`.

Expected outcome.

- `lake build` completes with zero errors (≈45–70 min cold build on a modern laptop; dominated by `native_decide` tables).
- `cook_universality_discharged` typechecks with zero `sorry`.
- `rule110_turing_universal_from_cook` typechecks with zero `sorry`.
- `#print axioms rule110_turing_universal_from_cook` lists exactly five Cook bridge axioms (plus standard Lean axioms and `native_decide` entries); no additional `sorry`.
- `#print axioms cook_c2_tape_bit_general` and `cook_cts_step_sim_ax` show only standard Lean axioms.

Reviewer verification protocol

The certification status is reproducible from the public repositories:

```
# 1. Clone and build rule110-lean
git clone https://github.com/novaspivack/rule110-lean
cd rule110-lean
lake exe cache get
lake build

# 2. Axiom audit on top theorems
lake env lean axiom_check.lean
```

```
# 3. Spot-check discharged theorems (Lean REPL or #check)
#    Rule110.CookLen6TailOrigin
#    Rule110.CookC2GeneralC1

# 4. Python cross-check (from ugp-physics checkout)
cd ../ugp-physics/papers/30_cook_theorem
python3 scripts/len6_evolved_origin_cert.py
```

Pin the git commit hash in `REPRODUCE.md` when updating the certification snapshot.

E Proof Sketches

The following proof sketches make the load-bearing formal arguments verifiable within this manuscript. Full machine-checked proofs for discharged theorems have zero `sorry` in `rule110-lean`.

E.1 Icc locality (`infRule110Steps_agree_Icc`)

Rule 110 has radius 1; n steps depend only on initial bits in $[i-n, i+n]$. Induction on n with the local update definition gives agreement of `infRule110Steps` when initial tapes agree on that interval. This lemma is the universal tool for replacing global tape equality with boundary-region hypotheses.

E.2 Support and data cone disjointness

Support gliders (ossifier and leader) are placed at origins `cts_ossifier_origin` and `cts_leader_origin`, far to the right of data slots $0, \ldots, 20$. For the empty word, `cts_support_agrees_on_data_cone` shows `cts_to_rule110_tape_with_support` and bare `cts_to_rule110_tape` agree on every index within ± 30 of each data slot origin; `native_decide` discharges the finite case split over slots. The generalized lemma `cts_support_agrees_on_data_cone_gen` supports transport of list-sim readback through support overlays for bounded words.

E.3 General C1 via glider spacing (`cook_c2_tape_bit_general`)

The key arithmetic: C2 gliders for different data slots are placed at spacing $\Delta = 42$ cells (one period of the ether $\times 3$), so slot s occupies tape positions $[1000 + 42s,\, 1000 + 42s + 5]$. A 30-step backward cone at slot s' covers $[1000 + 42s' - 30,\, 1000 + 42s' + 30]$. For $s \neq s'$ the glider at s intersects the cone at s' iff $|1000 + 42s - (1000 + 42s')| < 30 + 6 = 36$, i.e. $42|s - s'| < 36$. Since $42 > 36$, this never holds.

Module `CookC2GeneralC1` formalizes this argument: `cts_word_cell_slot_range` deduces which slot each cell comes from; `cts_word_cell_in_cone_is_readSlot` proves that any cell inside slot s''s cone must originate from s'; and `cts_to_rule110_tape_cone_eq_min_word` combines these to show the CTS tape for an arbitrary word w agrees with the single-slot min-word tape on the cone. Applying Theorem 3.1 and `cook_c2_tape_bit_min_word` yields `cook_c2_tape_bit_general`.

The final sub-case — when k falls inside the active C2 glider range $[c2\mathrm{SimOrigin}\, s',\, c2\mathrm{SimOrigin}\, s' + 5]$ with $w[s'] = \mathrm{true}$ — is discharged via `overrideCells_singleton_at` (`Gliders.lean`): both cell lists contain the identical C2 glider cell, and the membership instantiation closes. The proof carries zero `sorry` ($\mathbf{A}_{\mathrm{Lean}}$).

E.4 Bare equivalence for ossifier readback

For $L \leq 7$, `CookC2SupportBareEquiv` proves that overlaying the left ossifier in bounded simulation does not change read bits at data slots: `c2SimReadAtWithOssifier_eq_bare` and `c2SimRun_withOssifier_eq_bare_at_origin`. The proof combines (i) cone disjointness between ossifier patches and data read cones, (ii) Theorem 3.1 on the InfTape side, and (iii) `infRule110Steps_agree_Icc` to transport list certificates. Hence exhaustive bare-word checks imply support-encoded InfTape readback without separate init-cone `native_decide` on the full support tape.

E.5 $L = 6$ tail-origin certificate (validity)

Problem. For `cook_min_len6_cts` and input `[true]`, tail-origin after one appendant microstep requires equality at slot origins between (i) chunked InfTape evolution `infRule110Steps` $30 \circ$ `infRule110Steps` 390 on the phased encode and (ii) the mid-path list simulation after 420 bounded steps on `len6TruePhasedSupportInit`.

List certificate (discharged in Lean). Predicate `len6Tail420ComposeOriginOk` evaluates only `c2SimRun` and `List.getD` at six origins; `native_decide` completes in seconds. This is sound relative to infinite-tape semantics because `c2SimRun_eq_infRule110Steps_at` identifies list and InfTape values whenever the dependency cone fits in the prefix (`c2SimBound` $= 2500$, origins $\leq 1210 + 30$).

Why InfTape is not unfolded at $n = 390$. Applying `c2SimRun_eq_infRule110Steps_at` with $n = 390$ inside the proof kernel triggers normalization of 390 nested `infTapeStep` layers on the phased encode. Even at a single concrete cell index, elaboration exceeds Lean's default heartbeat threshold. Monolithic `native_decide` on `infRule110Steps` 420 is worse: each check rebuilds the entire phased tape as an `InfTape`. Chunking $(390 + 30)$ matches Cook's block structure and keeps the *checked* predicate list-based.

InfTape bridge (discharged in Lean). Theorem `len6_evolved_inf30_eq_list420_at_slot` is proved via the hard-coded literal `len6Evolved390` (`native_decide` in `CookLen6Evolved390Literal`) plus a 30-step `infRule110Steps_agree_Icc` bridge at slot origins.
Theorem `cook_cts_tail_origin_len6_M390_M30` ($\mathbf{A}_{\text{Lean}}$, zero `sorry`) follows.

Python cross-check. Script `len6_evolved_origin_cert.py` reimplements `c2SimStep` / `c2SimRun` with the same Rule 110 table and ether padding, loads the init tape exported from Lean (`export_len6_phased_init.lean`), and verifies the six origin equalities. The output JSON records `init_tape_sha256` and per-slot bits so third parties can reproduce the check without the Lean toolchain. Dual agreement (Lean `native_decide` + Python) addresses implementation risk on the list-side certificate; the InfTape transport is discharged in Lean, not assumed as an axiom.

References

[1] Matthew Cook. Universality in elementary cellular automata. *Complex Systems*, 15(1):1–40, 2004.

[2] Matthew Cook. A concrete view of rule 110 computation. *Electronic Proceedings in Theoretical Computer Science*, 1:31–55, 2009. arXiv:0906.3248.

[3] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Artificial Intelligence*, pages 625–635. Springer, 2021.

[4] Turlough Neary and Damien Woods. P-completeness of cellular automaton rule 110. In *Automata, Languages and Programming (ICALP 2006), Part I*, volume 4051 of *Lecture Notes in Computer Science*, pages 132–143. Springer, 2006.

[5] Nova Spivack. Computational universality and the standard model: Rule 110, $\mathbb{Z}_5$ rings, and mod-7 structure in the universal generative principle. Zenodo, 2026. Paper P28 in the UGP Physics series. https://doi.org/10.5281/zenodo.20259513.

[6] Nova Spivack. Reflexive reality and ugp physics programmes. Research Portal, 2026. Catalogues of the Reflexive Reality programme (NEMS/MFRR) and the UGP Physics programme, its quantitative branch.

[7] Nova Spivack. rule110-lean: Lean 4 formalization of the rule 110 cook universality pipeline. https://github.com/novaspivack/rule110-lean, 2026. Machine-certified Cook universality proof pipeline in Lean 4.